

COMP603/ENSE600

Program Design & Construction / Software Construction – Assessments

2026 Semester 1



Contents

Introduction	2
Recommended GenAI Tools	2
Important dates	2
Overview of the Assessments	3
Project 1 (40%) - Requirements and Marking Guidelines	3
Project 2 (60%) - Requirements and Marking Guidelines	5
Policies and Common Requirements (Apply to Both Assignments)	7
Project Code and Documents Submission	8
Submission Format	9
Appendix	10

Introduction

With the rapid development of *Artificial Intelligence (AI)*, the nature of software development has changed dramatically. In previous years, programming courses primarily assessed whether students could independently write programs based on the knowledge taught in class. However, in the era of *Generative AI (GenAI)*, the ability to merely produce code is no longer a sufficient indicator of software engineering competence.

This course adopts an AI-native software engineering pedagogy. Students are encouraged to use modern AI-assisted development tools (e.g., ChatGPT, Cursor, GitHub Copilot) and will be assessed on their ability to collaborate with AI, critique AI outputs, and make sound engineering decisions.

In this course, the focus of assessment has therefore shifted from code production to software design thinking and *Human-AI (HAI) collaboration*. Students are now expected not only to implement software systems, but also to demonstrate the ability to design, justify, critique, and refine system solutions with AI as an active development tool. GenAI is not treated as a shortcut or optional aid in this assignment; instead, it is a mandatory component of the development process and an essential part of the learning objectives.

This semester, you will design and develop a software product consisting of two related projects (Assignment 1 and Assignment 2) with the assistance of GenAI tools. Both assignments must be based on the **same project topic** and should represent a continuous development process, with Assignment 1 focusing on system design and **Java-CUI-based software**, and Assignment 2 extending the system with a **Java-GUI** and additional implementation requirements.

You may complete the project **individually or in a group of two students**. The marking rubric is identical for individual and group projects. Group work is strongly recommended, as collaborative development reflects real-world software engineering practice. For team projects, each member is expected to make a significant contribution to both design and implementation.

You may select a project topic from the recommended project categories and detailed project list provided (*see Appendix*) OR propose your own topic. Regardless of the topic, the primary goal of this assignment is not to build the largest or most complex system, but to demonstrate sound system modelling, critical design judgement, and effective HAI collaborative development.

Recommended GenAI Tools

- Microsoft Copilot (Access via AUT account) <https://copilot.microsoft.com/>
- OpenAI ChatGPT (Need register yourself) <https://chatgpt.com/>
- Google Gemini (Need register with a Google account) <https://gemini.google.com/>
- Anthropic Claude (Need register with a Google account) <https://claude.ai>

Important dates

- Register your group by the end of **Week 3** by **joining a group from Canvas**.
- Project 1 submission: **Week 7, by Friday 11:59 pm**
- Project 2 submission: **Week 13, by Friday 11:59 pm**

Overview of the Assessments

- There are two assessments. The first and second take **40%** and **60%**, respectively. Each assessment comprises lab exercises and a software project. Refer to the table below:

Assessment Item		Due Date
Assessment 1 40%	Lab Exercise Completion W1-W7 (15%) Complete at least 3 Labs out of 6	Weekly
	Software Project 1 (85%)	Week 7
Assessment 2 60%	Lab Exercise Completion W8-W11 (10%) Complete at least 2 Labs out of 4	Weekly
	Software Project 2 (90%)	Week 13

Where “**complete**” means you will need to **attempt to complete** the task and upload it to Canvas.

Late submission of labs is not counted. For semesters with different structures:

- 6 Weeks + Break + 6 Weeks:
 - First assessment for LABS: Covers Weeks 1-7
 - Second assessment for LABS: Covers Weeks 8-11
 - 7 Weeks + Break + 5 Weeks:
 - First assessment for LABS: Covers Weeks 1-6
 - Second assessment for LABS: Covers Weeks 8-11
- To pass the course**, students must satisfy the stated learning outcomes and achieve a minimum **overall grade of C- (50%)**.
 - IDE:** Students may use ANY IDE or development environment of their choice (e.g., NetBeans, IntelliJ, VS Code, Cursor, Eclipse, AI-Augmented editors, GenAI plugins) during the development process. However, the **final submission must be a fully functional NetBeans Ant or Maven project**. The responsibility for converting or importing the project into NetBeans format lies with the student.
 - JDK Version requirement: JDK 25**
 - Lab Exercise:** Students are encouraged to actively use **GenAI tools** to support the completion of all the lab exercises. The purpose of the labs is not only to practise Java programming, but also to develop the ability to collaborate effectively with AI, including writing good prompts, refining AI-generated code, and using AI to diagnose and fix problems. Students are expected to:
 - understand the code produced by GenAI,
 - be able to explain how it works,
 - and interact with GenAI to debug, improve, and adapt the solution when issues arise.

The labs, therefore, serve as a training ground for **AI-assisted software development**, helping students build confidence and competence in using modern development tools efficiently and responsibly.

Project 1 (40%) - Requirements and Marking Guidelines

This assignment focuses on software design thinking and HAI collaborative development. Students are required to propose a project topic, generate and analyse requirements, design the system

architecture, and implement a CUI-based Java Application using Generative AI tools.

Component 1.1: Lab Exercises (15%)

Students are required to **complete at least 3 of the 6 lab** exercises to demonstrate engagement with the core programming concepts taught in the course.

Component 1.2: Requirement Generation and Analysis (20%)

Students are required to:

- Propose a project topic from the project categories or created themselves.
- Ensure the project is neither too simple nor too complex.
- Generate and define the system requirements, including core functionalities, system scope, and user interactions.

Student-Authored Critical Reflection (No AI-Generated Text):

Students must critically articulate:

- Which features and requirements are essential for the system.
- Which features are unnecessary or unrealistic.
- Why these design decisions were made.

Component 1.3: Software Design (30%)

Based on the requirements, students are required to:

- Design the functionality using case diagrams.
- Define the system's scope, constraints, and assumptions if you have.
- Use GenAI to critically analyse the design and consider its suggestions.
- Produce software class diagrams showing classes, attributes, methods, and relationships.
- Evaluate and revise the design iteratively with the assistance of Generative AI tools.
- Include at least two iterations of the design, showing how the system architecture evolved.

Student-Authored Critical Reflection (No AI-Generated Text):

Students must critically reflect on:

- The design suggestions generated by GenAI.
- The errors, limitations, or poor design decisions made by GenAI.
- How these designs were corrected or improved.
- Why the final design is more appropriate.

Component 1.4: Software CUI Implementation (30%)

Based on the final design, students are required to:

- Implement the complete system using Java.
- Fully apply OOP concepts and make the system easy to understand and modify.

- Achieve abstraction using interfaces, abstract classes, and concrete classes.
- Ensure the Java application opens in NetBeans IDE, compiles, and runs successfully.
- Ensure the program is robust and free of manual configuration.
- Implement the designed classes and relationships (with the assistance of GenAI).
- Include a working CUI interface.
- Use suitable Java collections and use File I/O for data persistence.
- Use GenAI tools to assist with coding.
- Apply SOLID design principles.
- Demonstrate modularity and reasonable code structure.

Component 1.5: AI Usage Log and Demonstration Video (5%)

Students are required to:

- Include an *AI Usage Log in the appendix*, summarising main prompts used, types of tasks GenAI assisted with, and which parts were mainly human-designed.
- Submit a demonstration video (within 5 minutes) (1) explaining how GenAI was used, (2) demonstrating the system functionality, and (3) outlining the overall code structure (no detailed line-by-line walkthrough required).

Project 2 (60%) - Requirements and Marking Guidelines

Project 2 is a **direct extension of Project 1**. Students continue developing the same software system by adding a **Graphical User Interface (GUI), database, unit testing, and stronger architectural design**.

If a student or group wishes to change to a different project topic for a special reason, the change **must be approved by the lecturer** before any work begins. Unapproved topic changes may result in penalties or rejection of the submission.

Core System Requirements

- Be based on the **same project topic** as Project 1 (unless lecturer approval is granted).
- Be a **significant extension** of the Project 1 system.
- Be implemented as a **Java GUI application** (e.g., Swing or JavaFX).
- Be backed by an **Apache Derby Embedded** database accessed via **JDBC or Hibernate**.
- Be built and executed as a **NetBeans Ant or Maven project** without manual configuration.
- Use **Git/GitHub** to track development progress.
- Include a **unit testing layer** using the **JUnit framework** with meaningful test cases.
- Apply **software design patterns** (e.g., MVC, DAO, Singleton, Factory, Observer, etc.) where appropriate.
- Demonstrate **clear separation of concerns** between GUI, business logic, and data access.
- Be developed using **Generative AI tools** (e.g., ChatGPT, Cursor, Copilot) together with **human judgment and critical evaluation**.

Component 2.1: Labs (10%)

- Students are required to complete **at least two of the four lab exercises** to demonstrate

engagement with core programming concepts.

Component 2.2: Graphical User Interface (GUI) (20%)

The GUI must demonstrate:

- **A clear, professional, and well-structured layout.**
- **Ease of use**, allowing users to understand and navigate the system without confusion.
- **Consistency** between GUI actions and underlying business logic.
- **Revisions to AI-generated UI flows** to ensure they match real user workflows and business rules.

Students are expected to **critically evaluate and refine** AI-generated UI designs rather than accept them blindly.

Component 2.3: Database (20%)

The database layer must:

- Use **Apache Derby in Embedded mode**.
- Use **JDBC or Hibernate** for database access.
- Use a **Data-Access layer** (e.g., DAO pattern).
- Ensure that **GenAI-generated database code has been reviewed, corrected, and verified** to work properly.
- The database must be essential to the system's functionality, not just a cosmetic addition.

Component 2.4: Software Functionality, Design, Implementation & Testing (40%)

2.4.1 Functionality & Usability

- Smooth execution **without manual configuration** (e.g., no manual DB startup, no manual JAR setup).
- **Sufficient functional complexity** appropriate to the project topic and team size.
- **Logical flow** between GUI screens.
- **Correct persistence behaviour** (data saved and retrieved reliably).
- **Graceful handling of invalid input**, with clear user feedback rather than crashes.
- **OOP** concepts must be fully reflected

2.4.2 Architecture

- A clear **layered architecture** (e.g., MVC or similar).
- Correct application of **design patterns** such as:
 - Factory
 - Singleton
 - Mediator
 - Observer
 - or other appropriate patterns

If GenAI proposes inappropriate patterns or poor design, students must **identify, correct, and improve** them.

2.4.3 Version Control (Git / GitHub)

- Include the **.git folder** in the project submission.
- Show **multiple commits**.
- Demonstrate **evolution over time**, including refactoring after AI-generated code.
- Provide a **GitHub repository link** for pair work.
- A single “final dump” commit will receive **low marks** in this section.

2.4.4 Testing (JUnit Framework)

- Use the **JUnit framework 4**.
- Include **at least five meaningful test cases**.
- Ensure tests cover:
 - database operations, and
 - key business logic.
- Tests should be understandable and correctly named.
- Test cases should be built around the main functionality.

Component 2.5: AI Usage Log and Demonstration Video (10%)

Students are required to:

- Include an **AI Usage Log in the appendix**, summarising main prompts used, types of tasks GenAI assisted with, and which parts were mainly human-designed.
- Submit a demonstration video (within 5 minutes) (1) explaining how GenAI was used, (2) demonstrating the system functionality, and (3) outlining the overall code structure (no detailed line-by-line walkthrough required).

Policies and Common Requirements (Apply to Both Assignments)

Late Submission

- A late submission penalty of **5% per day** will be applied, up to a maximum of **5 days**.
- Submissions later than **5 days after the due date will not be accepted** and will receive a mark of zero.
- If a valid **Special Consideration Application (SCA)** is approved, no late penalty will be applied within the approved SCA period.
- If a student submits both an SCA and the assignment before the original due date, the late submission will **NOT be graded**.

Unsatisfactory Submission

Submissions will be considered unsatisfactory, and penalties may apply if any of the following occur:

- The final submitted project source code cannot be opened or run using **NetBeans IDE**.
- The submission is not provided as a **ZIP file**.

- The submission does not follow the **required file naming convention**.

Academic Integrity and Plagiarism

- Plagiarism and academic misconduct are treated seriously and will be reported according to university policy.
- If the assignment is classified as plagiarism, **all assessment components (except labs) will receive a mark of zero**.
- Plagiarism includes, but is not limited to:
 - Download open-source projects from online sources, e.g., GitHub, and modify the program
 - Re-use past students' assignments with modifications
 - Re-use the assignment of other AUT courses, e.g., ADA, DSA, etc.
 - Buy assignments from any source

Use of Generative AI Tools

- Students are allowed and **encouraged** to use **ANY GenAI** tools for this assignment.
- However, students must provide an **AI Usage Log** and a **NON-GenAI critical reflection** as specified in the assignment requirements.
- Failure to provide transparent documentation of AI usage will result in losing marks.

Group Contribution

- For group projects, all members must clearly state their individual contributions.
- If a student contributes **less than 40%**, their final mark will be capped at **60% of the total project mark**. This may be applied after moderation.
- If a student makes **no contribution**, they will receive a mark of **zero**.

Demonstration Video

- A **demonstration video** must be submitted as part of the assignment.
- The video should demonstrate system functionality and provide a high-level explanation of the system structure.
- If the video is missing, the team may be required to attend a **live review session (e.g., via MS Teams)**.
- Failure to submit a video AND failure to attend the review session will result in a **DNC (Did Not Complete)** for the assignment.

Data Management and Backup Responsibility

- Students are responsible for **maintaining and backing up multiple versions** of their programs throughout the project.
- It is strongly recommended to use **cloud storage services** such as OneDrive, Dropbox, or Google Drive for backup.
- Git/GitHub is optional for Project 1 but mandatory for Project 2.
- **Loss of data, files, or source code will not be accepted as a valid reason** for special consideration or deadline extension.

Project Code and Documents Submission

- The project must be submitted **via Canvas** by the due date.
- For group projects, **only one team member** needs to submit the project on behalf of the group.

- The submissions required below must consist of **one compressed ZIP file** containing the following items:

1. Project Source Code Folder (Components 1.4, 2.2, 2.3 and 2.4)

- The complete project folder, including all source code and related files
- The submitted project (source code) must be a complete and runnable **NetBeans project**.

2. Project Report (Word or PDF)

The report must include the following sections:

- Account credentials, if applicable (e.g., username and password).
- Project requirements (**Component 1.2**).
- Program design (**Component 1.3**).
- AI Usage Log in the appendix (**Components 1.5 and 2.5**).
- Contribution report, clearly stating the contribution of each team member (**for group projects only, Components 1.5 and 2.5**).

For group projects:

- If a team member contributes **less than 40%**, their final mark may be capped or penalised.
- If a team member makes **no contribution**, they may receive a mark of zero or fail the course.

3. Demonstration Video (Components 1.5 and 2.5)

- See requirements for Components 1.5 and 2.5

Submission Format

- The entire submission must be compressed into a **single ZIP file**.
- Submissions in other formats (e.g., multiple files, folders, or non-ZIP archives) may be considered unsatisfactory (**-5%**).
- The project must:
 - open directly in **NetBeans** without manual configuration (including adding libraries, starting Derby DB Services, setting up data, etc.),
 - compile successfully,
 - and run as intended using the standard **NetBeans** build and run process.
- Submissions that cannot be opened or executed in NetBeans may be considered unsatisfactory and subject to a penalty of **5% to 10%**.
- The ZIP file must follow the required **file naming convention** as specified below.
 1. Find your group ID from Canvas.
 2. For example, if the group ID is 9, and the student ID is 1234567, then the name of the submitted file should be “P09_1234567.zip”. If you have a partner in your group with student ID 7654321, the file name should be “P09_1234567_7654321.zip”.

Appendix

Recommended Topic Categories

Category	Name	Description
A	Academic & University Systems	Information systems that model academic processes and workflows within a university context, such as student records, course selection, project registration, and academic advising. These systems typically involve multiple user roles, structured data management, and business rules.
B	Business & Enterprise Systems	Enterprise-style information systems that support organisational operations such as customer support, inventory management, workforce planning, and transaction processing. These projects emphasise system workflows, data persistence, and modular system design.
C	Planning & Recommendation Systems	Systems that provide decision support or personalised recommendations based on user profiles, constraints, or predefined rules. These projects require modelling of logic, constraints, and reasoning processes rather than simple data storage.
D	Knowledge & Data Systems	Systems that focus on organising, storing, searching, and retrieving information or knowledge. These projects emphasise data modelling, indexing, classification, and information retrieval mechanisms.
E	Simulation & Control Systems	Systems that simulate the behaviour of real-world processes or control dynamic systems over time, such as traffic lights, elevators, hospital workflows, or queue systems. These projects require modelling of states, events, and transitions.
F	Games & Interactive Systems	Interactive systems based on explicit rules, game logic, and state transitions, including board games, card games, RPG games, and puzzle-based systems. These projects emphasise rule modelling, state management, and user interaction.
G	AI-era Systems (Advanced / Optional)	Systems that explicitly involve human-AI interaction, such as AI-assisted tools, chatbot evaluation platforms, and AI decision-explanation systems. These projects emphasise system integration, critical evaluation of AI outputs, and human-in-the-loop design.

Recommended Project Topics

Category A: Academic & University Systems

- Student Information Management System
- Course Selection System
- Virtual Academic Advisor
- Software Project Registration System
- Conference Management System
- Knowledge Repository
- Q/A System for Students and Lecturers

Category B: Business & Enterprise Systems

- Simple ERP System
- Service Desk System
- Workforce Planning System
- Inventory Management System
- Online Shopping System
- Ticket Booking System
- Hotel Booking System

Category C: Planning & Recommendation Systems

- Study Pathway Recommendation System
- Career Path Planner
- Fitness Training Planner
- Diet & Meal Planning System
- Travel Itinerary Planner
- Personal Budgeting Advisor
- Foreign Language Learning Software

Category D: Knowledge & Data Systems

- Knowledge Repository
- News Aggregation System
- Document Management System

- Resume Management System
- Research Paper Management System
- Survey & Polling Platform

Category E: Simulation & Control Systems

- Smart Home Simulator
- Traffic Light Control System
- Elevator Control System
- Hospital Patient Flow Simulation
- Restaurant Ordering & Kitchen Workflow
- Airport Check-in Simulation
- Theme Park Queue Simulation

Category F: Games & Interactive Systems

- Card Game
- Chess Game
- RPG Game
- Board Games
- Virtual Pet Game
- Escape Room Puzzle Game
- Deal or No Deal
- Who Wants to Be a Millionaire

Category G: AI-Era Systems (Advanced / Optional)

- AI-assisted Study Companion
- AI Prompt Management System
- Chatbot Evaluation System
- AI Recommendation Explanation System
- Human–AI Task Allocation System